

Laborator 1 Arhitecturi Hardware Reconfigurabile

Descrierea, simularea și implementarea circuitelor logice utilizând medii software de proiectare (Active HDL 7.1. și instrumente din pachetul Xilinx ISE 8.0)

Conținut laborator:

- 1. Obiectivul lucrării.**
- 2. Noțiuni teoretice:**
 - a. Sisteme de reprezentare a numerelor. Transformări între sisteme de numerotare (recapitulare);**
 - b. Circuite logice combinaționale (recapitulare);**
 - c. Etape în proiectarea circuitelor logice utilizând medii de proiectare software.**
- 3. Cerințe laborator.**
- 4. Temă.**

1. Obiectivul lucrării:

Laboratorul se constituie ca un îndrumar pentru lucrul cu mediul de proiectare, simulare și implementare ActiveHDL pentru structuri de calculatoare numerice. Operarea cu acest mediu de dezvoltare presupune o serie de etape de lucru, etape care vor fi prezentate detaliat într-o anexă, care va putea fi utilizată pe parcursul laboratoarelor ulterioare. Tot în această anexă se va găsi și o prezentare a machetei de laborator care va fi folosită la implementări cu descrierea pinilor utilizați. Pentru laboratoarele alocate disciplinei Calculatoare numerice sunt necesare cunoștințe dobândite din cursuri anterioare despre circuitele logice combinaționale și secvențiale, proiectarea circuitelor logice și moduri în care datele sunt reprezentate în circuite logice. De aceea, am adăugat la acest laborator și un scurt breviar teoretic recapitulativ, referitor la subiectele sus menționate.

Utilitatea laboratorului: Toate etapele prezentate în lucrul cu mediul Active HDL vor fi utilizate și în celelalte laboratoare deci este esențială înțelegerea acestora. Parcurgerea etapelor pentru proiectarea, simularea și implementarea circuitelor este obligatorie exact în ordinea în care sunt prezentate în cele ce urmează. Orice abatere de la această ordine va avea drept urmare apariția unor erori structurale sau funcționale. Pe de altă parte, mediile de proiectare pentru circuite logice sunt utilizate, la ora actuală, pe scară largă în dezvoltarea profesionistă a circuitelor integrate logice pe care le întâlnim în componența calculatoarelor. Cunoașterea acestora este necesară pentru o viitoare carieră de inginer în electronică.

2. Noțiuni teoretice

- a. Sisteme de reprezentare a numerelor. Transformări între sisteme de numerotare**

Încă din școala primară noi suntem familiarizați cu **sistemul de numerotare zecimal (în baza 10)**. Acesta utilizează zece cifre (0,1 ... 9) cu ajutorul cărora se poate construi orice număr. Sistemul de numerotare zecimal este utilizat în viața noastră de zi cu zi dar și în aproape toate domeniile științelor exacte: matematică, fizică, chimie etc.

În circuitele logice, care sunt componente de bază ale oricărei structuri dintr-un calculator, se utilizează **sistemul de numerotare binar (în baza 2)**. Sistemul de numerotare binar utilizează doar două cifre (0 și 1) pentru reprezentarea oricărui număr. Explicația utilizării acestui sistem de numerotare este una simplă: se poate implementa ușor utilizând contacte electronice, așa cum este reprezentat în figura 1. Acest sistem poate fi utilizat cu ușurință pentru transmiterea datelor pe magistrale compuse din unul sau mai multe fire (prezență tensiune pe fir : 1 logic, absență tensiune : 0 logic) precum și pentru stocarea (memorarea) datelor în sisteme electronice (prezență sarcină, lipsă sarcină).

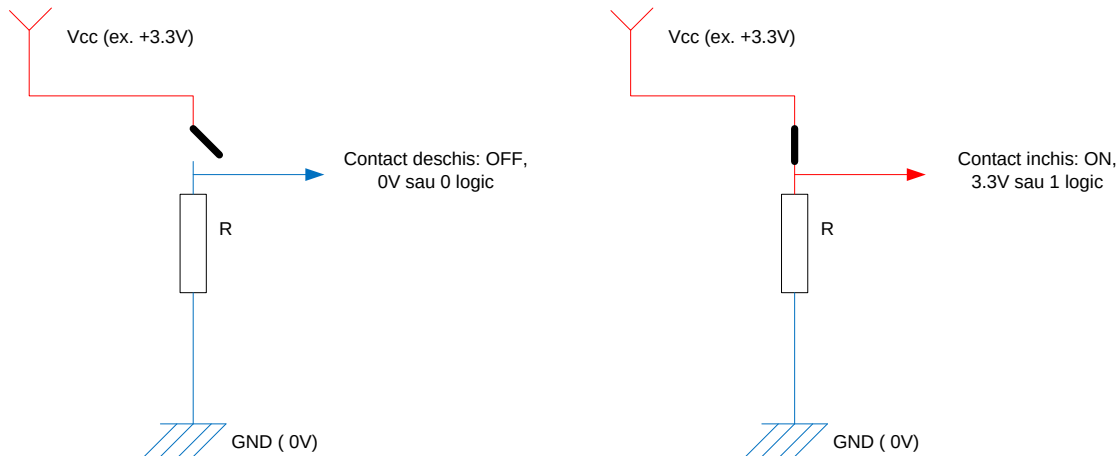


Fig.1. Reprezentarea binară a numerelor utilizând contacte electronice

Datorită utilizării frecvente a sistemului de numerotare binar în calculatoare, s-a introdus o denumire specială care se referă la cea mai mică cantitate de informație care se poate reprezenta printr-un număr și anume bitul. Bitul reprezintă de fapt un număr binar (în baza 2) compus dintr-o singură cifră. Este utilizat ca unitate de măsură în aproape toate subdomeniile calculatoarelor: lățimea unei magistrale de date (câte fire pe care pot circula valori 0 sau 1 poate avea o magistrala), viteza unei transmisii de date (câte cifre de 0 sau 1 pot fi transmise pe un fir într-o secundă), spațiul de memorare disponibil (câte celule care pot stoca 0 sau 1 are o memorie), dimensiunea numerelor (câte cifre de 0 sau 1 pot fi utilizate pentru reprezentarea unui număr) etc.

Alături de bit se utilizează și multiplii ai acestuia, prezentați în tabelul de mai jos:

Tabelul 1:

Denumire multiplu	Număr de biți
Nibble (digit)	2^2
Byte (B - octet)	2^3
Word (W - cuvânt)	2^4
Kilobit (kb)	2^{10}
Megabit (Mb)	2^{20}
Gigabit (Gb)	2^{30}
Terabit (Tb)	2^{40}

Ca extindere a reprezentării numerelor în binar, se utilizează **sistemul de numerotare hexazecimal (în baza 16)**. Orice număr se reprezintă aici utilizând digiți (câte 4 biți). Fiecare valoare distinctă a unui digit are alocat câte un simbol, cifre și litere: așa cum se poate vedea în tabelul de mai jos:

Tabelul 2:

Număr zecimal	Număr binar (digit)	Număr hexazecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	0001 0000	10

Transformarea unui număr zecimal într-un număr binar se face prin împărțiri succesive la 2. Se vor lua resturile împărțirilor, în ordine inversă.

De exemplu, pentru a determina valoarea numărului exprimat în zecimal 1234 în binar se fac împărțiri la 2 ca în tabelul de mai jos:

		Rest
1234	:2 = 617	0
617	:2 = 308	1
308	:2 = 154	0
154	:2 = 77	0
77	:2 = 38	1
38	:2 = 19	0
19	:2 = 9	1
9	:2 = 4	1
4	:2 = 2	0
2	:2 = 1	0
1	:2 = 0	1

Numărul rezultat este 100 1101 0010b adică toate resturile citite în sens invers.

Transformarea unui număr binar într-unul zecimal se face înmulțind cu puterea lui 2 fiecare cifră din numărul binar, de la dreapta la stânga.

Exemplu, numărul 1 0001 0110b se transformă în zecimal astfel:

$$1\ 0001\ 0110b = 0\ 2^0 + 1\ 2^1 + 1\ 2^2 + 0\ 2^3 + 1\ 2^4 + 0\ 2^5 + 0\ 2^6 + 0\ 2^7 + 1\ 2^8 = 2 + 4 + 16 + 256 = 278.$$

Extrem de utile sunt **transformarea binar – hexazecimal** și **transformarea hexazecimal – binar**.

Prima se face urmărind următorii pași:

1. Se desparte numărul în binar în grupe de câte 4 cifre (digiți) de la dreapta la stânga.
2. Fiecare digit se reprezintă, conform tabelului 2, cu un simbol din hexazecimal.

Exemplu: dacă avem numărul binar 10010110111b pentru reprezentarea lui în hexazecimal se procedează astfel:

$$10010110111b = 100\ 1011\ 0111b = 4B7h$$

Transformarea hexazecimal – binar se face astfel:

1. Fiecare simbol hexazecimal (cifră sau literă) se înlocuiește cu un digit (4 biți) binar, conform tabelului 2.

Exemplu: pentru numărul A50B6h avem:

$$A50B6h = 1010\ 0101\ 0000\ 1011\ 0110b$$

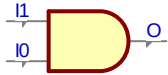
Transformarea zecimal – hexazecimal și **hexazecimal – zecimal** se face similar cu cea din zecimal în binar respectiv din binar în zecimal dar, de data asta prin împărțiri la 16 sau prin adunări între puteri ale lui 16.

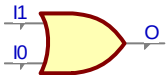
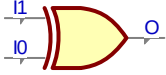
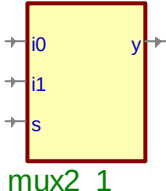
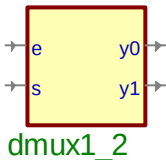
În aceste laboratoare, numerele în binar vor fi însoțite de litera b (ex. 1001b) iar numerele în hexazecimal de litera h (ex. 10A8h).

b. Circuite logice combinaționale

În continuare sunt prezentate câteva circuite combinaționale elementare a căror cunoaștere este obligatorie pentru însușirea cunoștințelor prezentate pe parcursul laboratoarelor:

Tabelul 3:

Denumire	Simbol	Descriere	Descriere VHDL
Poarta și (AND, SI)		Cunoscut sub denumirea de minim. Dacă una din cele două intrări este 0 atunci ieșirea este 0. Când ambele intrări sunt 1	$o \leq i1 \text{ and } i0;$

		ieșirea este 1	
Poarta sau (OR, SAU)		Cunoscut sub denumirea de maxim. Dacă una din cele 2 intrări este 1 atunci ieșirea este 1. Când ambele intrări sunt 0 ieșirea este 0.	$o \leq i1 \text{ or } i0;$
Poarta sau exclusiv (XOR, SAU EX)		Cunoscut și sub denumirea de non – echivalență. Dacă cele două intrări sunt egale (0 sau 1) atunci ieșirea este 0 , daca sunt diferite ieșirea este 1	$o \leq i1 \text{ xor } i0;$
Poarta inversoare (INV)		Ieșirea este negatul (CC1) intrării.	$o \leq \text{not } i;$
Multiplexorul cu 2 intrări și cu o ieșire (MUX2:1)		Dacă pe intrarea s (selecție) este 0 atunci pe ieșirea y va fi conectată intrarea i0. Dacă pe intrarea s este 1 atunci pe ieșirea y va fi conectată intrarea i1.	with s select $y \leq i0 \text{ when '0',}$ $i1 \text{ when '1',}$ $'0' \text{ when others;}$
Demultiplexorul cu o intrare și două ieșiri (DMUX1:2)		Dacă pe intrarea s (selecție) este 0 atunci ieșirea y0 va fi conectată la intrarea e în timp ce ieșirea y1 se va afla în starea inactivă. dacă pe intrarea s este 1 atunci y0 va fi inactivă iar pe ieșirea y1 se va conecta intrarea e.	process(e,s) begin if e = '1' then if s = '1' then y1 <= e; y0 <= '0'; else y0 <= e; y1 <= '0'; end if; else y0 <= '0'; y1 <= '0'; end if; end process;

Există circuite logice realizate din interconectarea între porți logice elementare care au deja denumiri consacrate: INV + AND = NAND (și negat), INV + OR = NOR (sau negat), INV + XOR = EQU (echivalență). Cu excepția inversorului, toate celelalte circuite pot avea un număr de intrări mai mare decât cel specificat în tabel dar funcționarea lor urmărește descrierea din tabel.

c. Etape în proiectarea circuitelor logice utilizând medii de proiectare software

Proiectarea unor circuite care să fie apoi implementate pe structuri hardware reconfigurabile, de tipul FPGA (Field Programmable Gates Array), se face prin utilizarea unor medii speciale de proiectare. De regulă, fiecare fabricant de circuite reconfigurabile furnizează și propriile instrumente software de sinteză și de implementare. Mediul de proiectare, simulare, implementare ActiveHDL realizat de firma Aldec permite selectarea instrumentelor software pentru sinteza și implementarea circuitelor logice în funcție de fabricant, astfel încât acesta poate lucra cu o clasă foarte mare de circuite reconfigurabile. De asemenea acesta are trei metode prin care pot descrie circuitele logice: prin limbaj de descriere hardware, prin editarea schematică a circuitului sau prin mașini de stare, pentru circuitele secvențiale. În prezentarea etapelor din anexa atașată laboratorului, vor fi exemplificate doar primele două metode de descriere, întrucât acestea vor fi utilizate în laboratoare.

După descrierea circuitului, mediul ActiveHDL permite simularea comportamentului în timp a acestuia, astfel încât este posibilă verificarea corectitudinii circuitului descris înainte ca acesta să fie procesat pentru implementarea în FPGA.

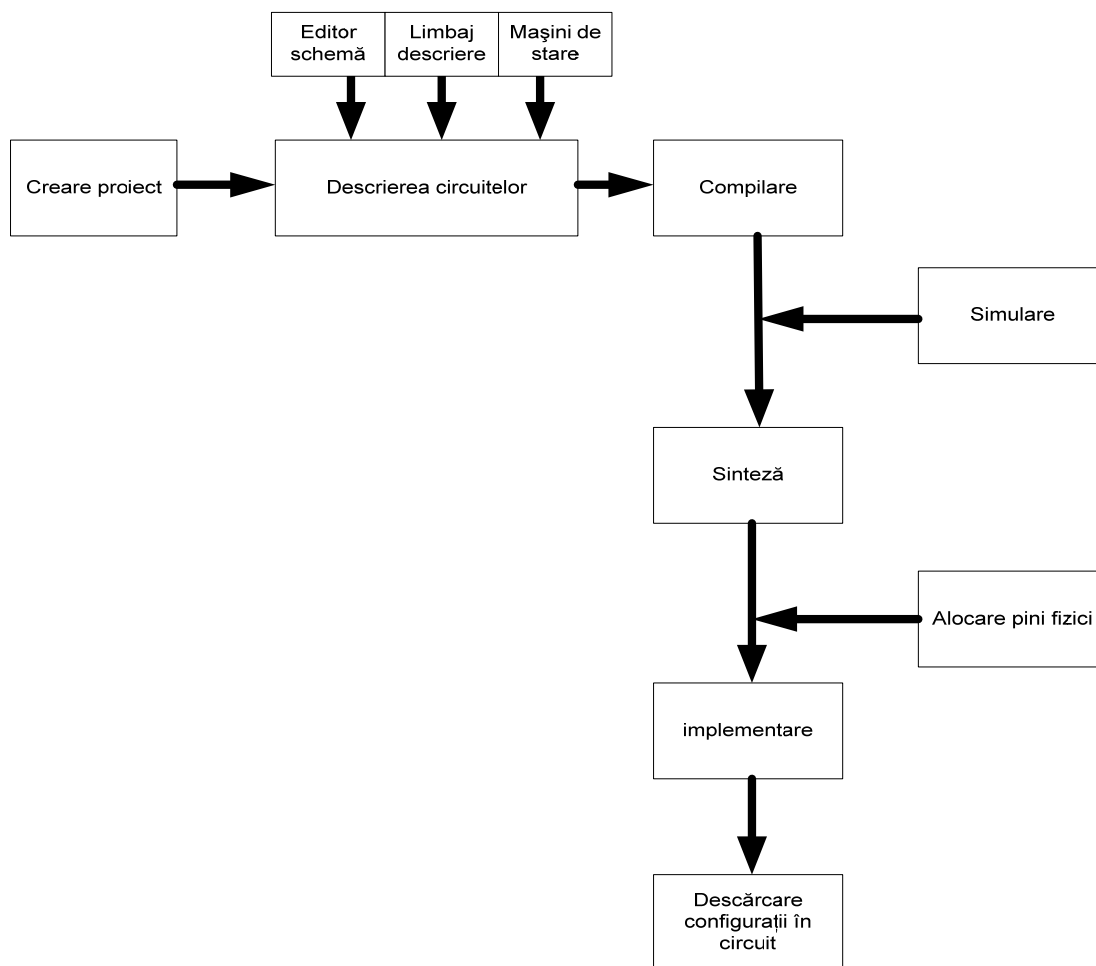


Fig.2. Etapele în proiectarea unui circuit logic utilizând un mediu software de dezvoltare

Sinteza și implementarea sunt etapele care permit configurarea circuitului reconfigurabil selectat. Funcționarea acestuia se testează folosind machetele de laborator.

Toate etapele prezentate în anexa atașată vor fi parcurse de-a lungul laboratoarelor pentru proiectarea unor arhitecturi de structuri de calculatoare numerice, simularea comportamentului acestora și implementarea lor.

O parte din etapele prezentate în anexă se vor referi la un exemplu de circuit: dar pașii parcurși, așa cum am amintit, sunt universali valabili, indiferent de tipul circuitului.

Etapa de simulare a fost prezentată în afara fluxului compilare – sinteză - implementare din figura 2 deoarece este una opțională. Etapa de alocare a pinilor fizici, este și ea teoretic una opțională, dar este necesară pentru a putea efectua experimentele dorite pe macheta de laborator.

Cerințe laborator:

1. Să se identifice circuitele periferice de pe machetă, prezentate în schema de la punctul F al anexei.
2. Parcurgând etapele descrise în anexă să se proiecteze, simuleze și implementeze un circuit de adunare a trei numere pe un bit (sumator complet pe un bit) având schema din figura 3.

Rezolvare:

2.a. Se va crea proiectul (punctul A din anexă). Se va descrie circuitul prezentat în figura de mai jos utilizând editorul schematic (punctul B). Se va compila (punctul C).

2.b. Se va realiza simularea (punctul D) astfel: pe portul a se va conecta ca stimul un semnal de ceas cu frecvența 8 MHz. Pe portul b se va conecta ca stimul un semnal de ceas cu frecvența 4 MHz. Pe portul ci se va conecta ca stimul tasta C. Se va simula comportamentul circuitului pentru $ci=0$ și $ci=1$ trecând prin toate stările intrărilor a și b.

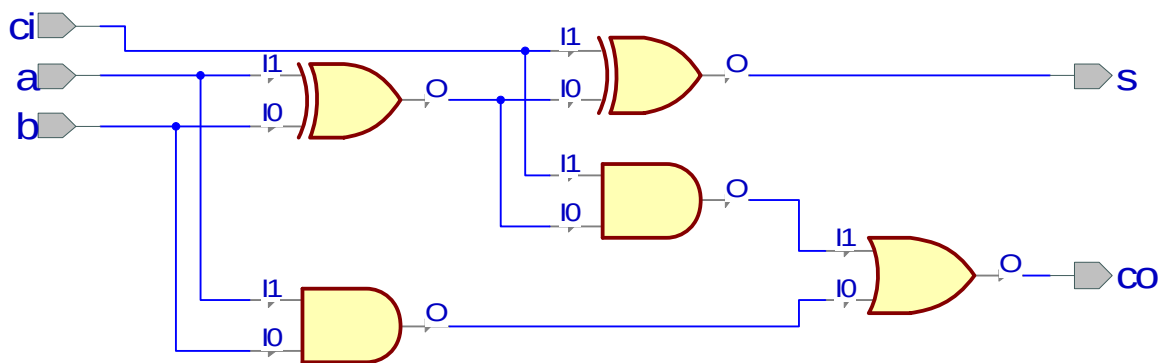


Fig.3. Sumatorul complet pe un bit – schema circuit

2.c. Se va realiza sinteza și implementarea circuitului pe macheta FPGA (punctul E) conectând porturile logice ca în tabelul de mai jos (tabelul de la punctul F):

Port logic	Circuit machetă
a	sw_dip0
b	sw_dip1
ci	sw_dip2
s	led0
co	led1

2.d. Se va descărca programul și se va opera cu el pe machetă.

3. Parcurgând etapele descrise în anexă să se construiască, utilizând descrierea în VHDL (limbaj de descriere hardware), un circuit multiplexor 4:1. Circuitul va avea următoarele porturi:

- 4 intrări, pe un bit fiecare, denumite i0, i1, i2 și i3;
- 1 intrare pe doi biți, denumită s;
- 1 ieșire pe un bit denumită y.

Să se sintetizeze și implementeze apoi circuitul, și să se descarce în machetă.

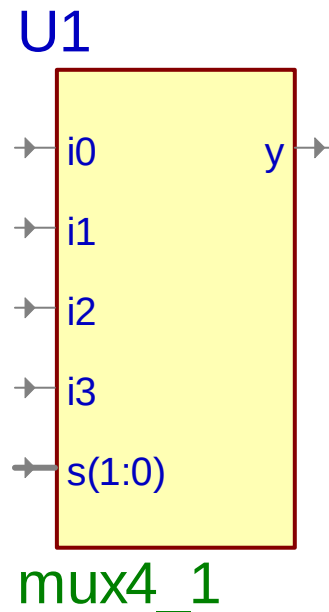


Fig.4. Multiplexorul 4:1

Rezolvare:

3.a. Se va crea un proiect (A). Se va crea, utilizând Wizard, un fișier VHDL pentru descrierea în limbajul VHDL, având porturile specificate în problemă. După crearea fișierului, se va adăuga în arhitectură (între begin și end) codul vhdL care descrie comportamentul circuitului multiplexor 4:1. Acesta este prezentat în listingul de mai jos cu explicații pentru fiecare instrucțiune.

Listing

with s select
y <= i0 when "00",

Explicație

Testează starea intrării s (pe 2 biți). Dacă intrarea s este:
- „00” atunci pe ieșirea y este conectată intrarea i0,

- i1 when "01",
 - i2 when "10",
 - i3 when "11",
 - 0 when others;
- „01” atunci pe ieșirea y este conectată intrarea i1,
 - „10” atunci pe ieșirea y este conectată intrarea i2,
 - „11” atunci pe ieșirea y este conectată intrarea i0,
 - altă valoare atunci ieșirea y este conectată la 0 (caz practic imposibil dar cerut de sintaxa VHDL).

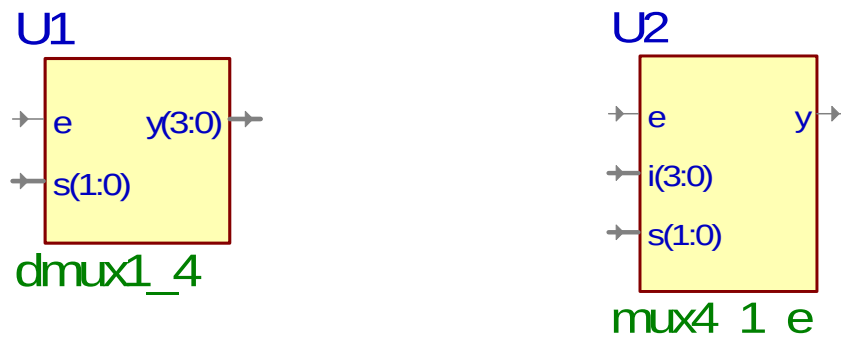
3.b. Se va sintetiza și implementa, porturile logice vor fi conectate ca în tabelul de mai jos:

Port logic	Circuit machetă
i0	sw_dip0
i1	sw_dip1
i2	sw_dip2
i3	sw_dip3
s[0]	sw_dip4
s[1]	sw_dip5
y	led0

4. Să se descrie utilizând limbajul VHDL și mediul ActiveHDL și să se implementeze pe macheta cu FPGA următoarele 2 circuite: un demultiplexor 1:4 și un multiplexor 4:1 cu validarea ieșirii.

Rezolvare:

Într-un proiect creat în ActiveHDL se vor crea două fișiere vhd: dmux1_4.vhd și mux4_1_e.vhd utilizând Wizard, așa cum se prezintă în anexă. Cele două circuite care se doresc create au următorii pini de intrare ieșire, prezentați mai jos:



a) demultiplexor 1:4

b) multiplexor 4:1 cu validare ieșire

Fig.5. Circuitele descrise la problema 4

Pentru demultiplexor entitatea se va numi tot dmux1_4 iar arhitectura dmux1_4_a iar pentru multiplexor entitatea se va numi mux4_1_e iar arhitectura mux4_1_e_a.

În fișierul dmux1_4.vhd, după ce acesta a fost creat cu Wizard, se vor scrie în interiorul arhitecturii (între begin și end) următoarele:

Listing

```
process(e,s)
begin
if e = '1' then
case s is
when "00" => y <= "0001";
when "01" => y <= "0010";
when "10" => y <= "0100";
when "11" => y <= "1000";
when others => y <= "0000";
end case;
else
y <= "0000";
end if;
end process;
```

Explicație

Bloc de comenzi secvențiale care se activează la schimbarea stării lui e sau a lui s

Dacă intrarea e este 1 logic atunci

Testează starea intrării s. Dacă intrarea s este:

- „00” atunci pe ieșirea y (4 biți) vom avea „0001” adică $y(0) = 1$;
- „01” atunci pe ieșirea y vom avea „0010” adică $y(1) = 1$;
- „10” atunci pe ieșirea y vom avea „0100” adică $y(2) = 1$;
- „11” atunci pe ieșirea y vom avea „1000” adică $y(3) = 1$;
- altă valoare decât cele mai de sus pe y vom avea „0000”.

Dacă intrarea e nu este 1 logic ($e = 0$) atunci

Ieșirea y este „0000”, deci toți biții de ieșire vor fi dezactivați adică 0 logic.

Sfârșit bloc de comenzi secvențiale.

În fișierul mux4_1_e, după ce acesta a fost creat cu Wizard, se vor scrie în interiorul arhitecturii (între begin și end) următoarele:

Listing

```
process(e,i,s)
begin
if e = '1' then
case s is
when "00" => y <= i(0);
when "01" => y <= i(1);
when "10" => y <= i(2);
when "11" => y <= i(3);
when others => y <= '0';
end case;
else
y <= 'Z';
end if;
end process;
```

Explicație

Bloc de comenzi secvențiale care se activează la schimbarea stării lui e, i sau s

Dacă intrarea e este 1 logic atunci

Testează starea intrării s. Dacă intrarea s este:

- „00” atunci pe ieșirea y vom avea conectată intrarea i(0);
- „01” atunci pe ieșirea y vom avea conectată intrarea i(1);
- „10” atunci pe ieșirea y vom avea conectată intrarea i(2);
- „11” atunci pe ieșirea y vom avea conectată intrarea i(3);
- altă valoare decât cele mai de sus pe y vom avea „0000”.

Dacă intrarea e nu este 1 logic ($e = 0$) atunci

Ieșirea y este în înaltă impedanță.

Sfârșit bloc de comenzi secvențiale.

Pentru implementarea pe machetă, se urmăresc etapele de sinteză și implementare prezentate în anexă. Alocarea pinilor se face astfel:

Pentru demultiplexorul 1:4 avem:

Port logic	Circuit machetă
e	sw_dip0
s(0)	sw_dip1
s(1)	sw_dip2
y(0)	led0
y(1)	led1
y(2)	led2
y(3)	led3

Pentru multiplexorul 4:1 cu validare avem:

Port logic	Circuit machetă
e	sw_dip0
s(0)	sw_dip1
s(1)	sw_dip2
i(0)	sw_dip3
i(1)	sw_dip4
i(2)	sw_dip5
i(3)	sw_dip6
y	led0

4. Temă

1. Să se transforme din hexazecimal în binar următoarele numere: 3A0Bh, 12345h, B10h.
2. Să se transforme din binar în hexazecimal următoarele numere: 1000110111b, 101011b, 1000000001111b.
3. Să se descrie, utilizând limbajul VHDL, un multiplexor 8:1. Se cer simulări ale circuitului utilizând mediul ActiveHDL.
4. Utilizând multiplexorul de la punctul 3 și multiplexorul 4:1 prezentat în laborator să se construiască (utilizând editorul schematic) un multiplexor 32:1. Se cer simulări ale circuitului, utilizând mediul ActiveHDL.

Criterii de evaluare:

Cerințe laborator	
1	1p
2	2p
3	2p
4	2p
Temă	
1	0,5p
2	0,5p
3	2p
4	3p